



ISPARTA
UYGULAMALI BİLİMLER
ÜNİVERSİTESİ

SIRALI MODELLER
RNN (RECURRENT NEURAL NETWORK)

AD: Mehmet Eren

NO:2412705019

GİRİŞ:

Geleneksel yapay sinir ağları, girdi verilerinin her birini birbirinden bağımsız birer birim olarak kabul eder. Ancak metin verileri, ses dosyaları veya zaman serileri gibi ardışık yapıdaki verilerde, bir verinin anlamı kendisinden önce gelen girdilere doğrudan bağlıdır. Yinelemeli Sinir Ağları (Recurrent Neural Networks - RNN), bu ardışık yapıdaki verileri işlemek ve verideki zamansal bağımlılıkları modellemek amacıyla geliştirilmiş bir derin öğrenme mimarisidir.

Konunun Neden Önemli Olduğu

Veri biliminde sıralı veri işleme yeteneği, bilginin sadece mevcut haliyle değil, geçmişteki bağlamıyla birlikte değerlendirilmesini sağlar. RNN mimarisinin önemi şu temel noktalara dayanmaktadır:

Bellek Mekanizması: RNN, gizli durum hidden state adı verilen yapısı sayesinde geçmiş zaman adımlarındaki bilgileri hafızasında tutarak mevcut tahmine dahil eder.

Doğal Dil İşleme (NLP): Cümle içerisindeki kelimelerin diziliş sırası anlamı doğrudan etkilediği için, metin analizi ve sınıflandırma işlemlerinde klasik yöntemlere göre daha başarılı sonuçlar üretir.

Duygu Analizi Uygulamaları: Bu projenin de temelini oluşturan metin tabanlı duygu analizi, kullanıcı eğilimlerini anlamak ve büyük veri kümelerini anlamlandırmak açısından kritik bir rol oynamaktadır.

1) Sequence (Ardışık Veri) Türleri

Derin öğrenme mimarileri, giriş ve çıkış verilerinin boyutuna ve dizilimine göre dört temel sınıfa ayrılır:

One-to-One (Bire Bir): Sabit boyutlu girdilerden sabit boyutlu çıktılar üretilir.

One-to-Many (Bire Çok): Tek bir girdiden dizesel bir çıktı üretilir.

Many-to-One (Çoka Bir): Zamansal veya dizesel veriler işlenerek tek bir çıktı elde edilir.

Many-to-Many (Çoka Çok): Girdinin ve çıktının birer dizi olduğu durumlardır.

2) Yinelemeli Sinir Ağları (RNN)

Yinelemeli Sinir Ağları, ardışık verilerdeki zamansal bağımlılıkları modellemek üzere tasarlanmıştır. Geleneksel ağlardan en temel farkı, bir hafıza mekanizmasına sahip olmasıdır.

Gizli Durumun Hesaplanması: Zaman adımı t 'deki gizli durum $h(t)$, bir önceki zaman adımındaki gizli durum $h(t-1)$ ve o anki girdi $x(t)$ kullanılarak şu şekilde ifade edilir:

$$h(t) = \tanh(W_{hh} h(t-1) + W_{xh} x(t) + b)$$

Burada W_{hh} gizli durumunun ağırlık matrisini, W_{xh} ise girdinin ağırlık matrisini ve b de sapma terimini temsil eder. Bu denklem sayesinde, ağ her adımda yeni bir girdi alırken önceki adımlardan gelen bilgiyi de günceller.

3) BPTT (Zaman İçinde Geriye Yayılım-Backpropagation Through Time)

RNN modellerinin eğitimi, standart geriye yayılım algoritmasının zaman eksenine genişletilmiş hali olan BPTT ile gerçekleştirilir. Model, her zaman adımındaki hatayı hesaplar ve toplam hatayı minimize etmek için ağırlıkları zaman adımları boyunca geriye doğru günceller.

Açılmış Ağ: BPTT uygulanırken RNN zaman adımları boyunca açılır ve sanki çok katmanlı tek bir ağırmış gibi işlem görür.

Ağırlık Paylaşımı: Ağ boyunca kullanılan ağırlık matrisleri (W) tüm zaman adımlarında aynıdır. Bu durum parametre sayısını azaltır ancak gradyanların hesaplanmasını karmaşılaştırır.

4) RNN Problemleri

Diziler uzadıkça BPTT süreci iki büyük matematiksel probleme yol açar:

Kaybolan Gradyan (Vanishing Gradient): Gradyanlar, zaman adımları boyunca geriye doğru çarpılarak ilerler. Eğer bu çarpanlar 1'den küçükse, gradyanlar üstel olarak küçülür ve sıfıra yaklaşır. Bu durumda ağ, cümlelerin başındaki kelimeleri (uzun süreli bağımlılıkları) unuttur.

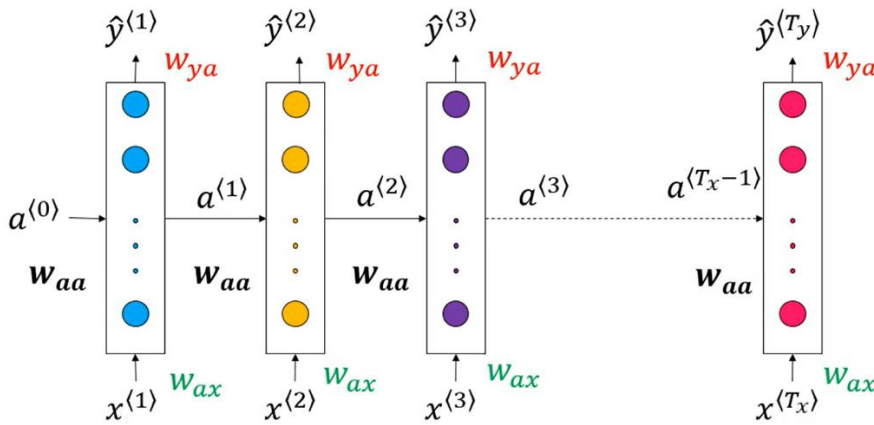
Patlayan Gradyan (Exploding Gradient): Eğer çarpanlar 1'den büyükse, gradyanlar üstel olarak büyüyerek sonsuza yaklaşır. Bu durum ağırlıklarının aşırı büyük değerlere ulaşmasına ve öğrenmenin bozulmasına neden olur.

5) Gradyan Kırpma (Gradient Clipping)

Patlayan gradyan problemini önlemek için kullanılan bir optimizasyon tekniğidir.

Eğer hesaplanan gradyan değeri, önceden belirlenmiş bir eşik değerin üzerindeyse, gradyanın değeri doğrudan bu eşığa indirgenir. Gradyanların büyüklüğü sınırlandırarak, ağırlık güncellemelerinin kararlı olması sağlanır. Bu, modelin eğitim esnasında sapmasını engeller.

YÖNTEM / MODEL AÇIKLAMASI



Şekilde görüldüğü üzere, model her zaman adımında aynı W_{ax} ve W_{aa} ağırlık matrislerini kullanır. Bu durum, modelin parametre sayısını azaltırken, verideki zamansal özellikleri konuma bağlı olmaksızın öğrenmesini sağlar.

Süreç, genellikle sıfır vektörü olan bir başlangıç aktivasyonu ($a^{(0)}$) ile başlar. Her adımda elde edilen ($a^{(t)}$) bilgisi, bir sonraki hücreye aktarılır. Bu sayede cümlelerin başındaki bir kelimenin etkisi, cümlelerin sonuna kadar taşınabilir. Her zaman adımında bir

Çıktı ($\hat{y}^{(t)}$) üretilir. Projemizde kullanılan Many-to-One yapısında, sadece son zaman adımındaki çıktı dikkate alınarak nihai duygu tahmini gerçekleştirilir.

UYGULAMA ALANLARI

Duygu Analizi (Sentiment Analysis): Bu projenin de odak noktasıdır. Müşteri yorumlarını, tweetleri veya ürün değerlendirmelerini otomatik olarak sınıflandırmak için kullanılır.

Makine Çevirisi: Bir dildeki kelime dizisini alıp, anlam bütünlüğünü koruyarak başka bir dildeki diziye dönüştürür (Örn: Google Translate).

Metin Üretimi: Bir başlangıç kelimesinden yola çıkarak mantıklı bir cümlelerin geri kalanını tahmin eder (Örn: Otomatik tamamlama sistemleri).

Finansal Tahminler: Geçmiş borsa verilerini ve fiyat hareketlerini analiz ederek gelecek trendlerini öngörür.

Hava Durumu Tahmini: Sıcaklık, basınç ve nem gibi değişkenlerin zamansal değişimlerini modelleyerek tahmin yürütür.

MİNİ PROJE AÇIKLAMASI

Bu mini proje kapsamında, Yinelemeli Sinir Ağları (RNN) kullanılarak metin tabanlı bir duygu analizi modeli geliştirilmiştir. Modelin temel amacı, verilen bir film yorumunun içeriğini analiz ederek "olumlu" veya "olumsuz" olarak sınıflandırmaktır.

Çalışmada IMDB Film Yorumları Veri Seti kullanılmıştır. Veri setindeki en yüksek frekanslı 10.000 kelime analize dahil edilmiştir. Girdi boyutlarını eşitlemek amacıyla her yorum 500 kelime ile sınırlandırılmış; kısa yorumlar için dolgu, uzun yorumlar için ise kesme işlemleri uygulanmıştır.

Çalışmada, ardışık verilerin işlenmesine olanak tanıyan **Many-to-One** RNN mimarisi kullanılmıştır. Model; bir kelime gömme (Embedding) katmanı, dizisel verideki bağımlılıkları öğrenen 32 birimli bir SimpleRNN katmanı ve sonucu olasılıksal bir değere dönüştüren tek nöronlu Dense katmanından oluşmaktadır.

Algoritma, Python programlama dili ve Keras API'si kullanılarak geliştirilmiştir. Modelin optimizasyonu için RMSprop algoritması seçilmiş, hata ölçümü için ise Binary Crossentropy fonksiyonu kullanılmıştır. Eğitim süreci 128'lik batch boyutları ile 5 epoch üzerinden tamamlanmıştır.

Yapılan son eğitim denemesinde, model 5. epoch sonunda **%84.18** gibi yüksek bir doğrulama başarısına (val_acc) ulaşmıştır.

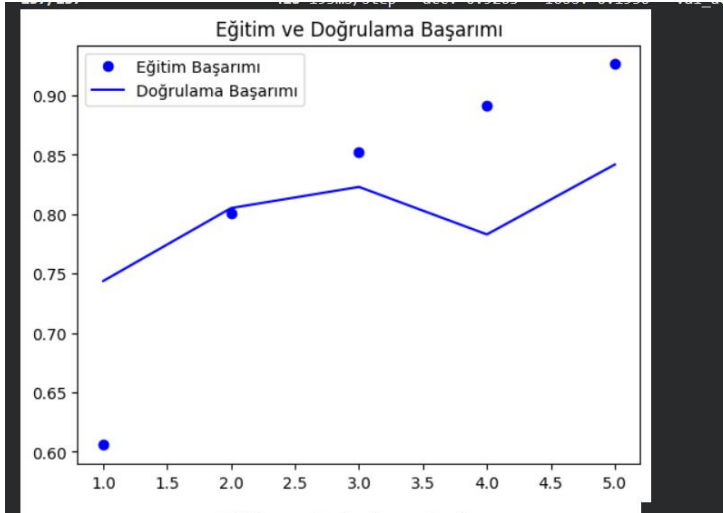
Yapılan son eğitim denemesinde, model 5. epoch sonunda %84,18 gibi yüksek bir doğrulama başarısına (val_acc) ulaşmıştır.

```
Kütüphaneler başarıyla yüklendi.  
Veriler yükleniyor...  
Veri ön işleme tamamlandı.  
Model: "sequential_2"
```

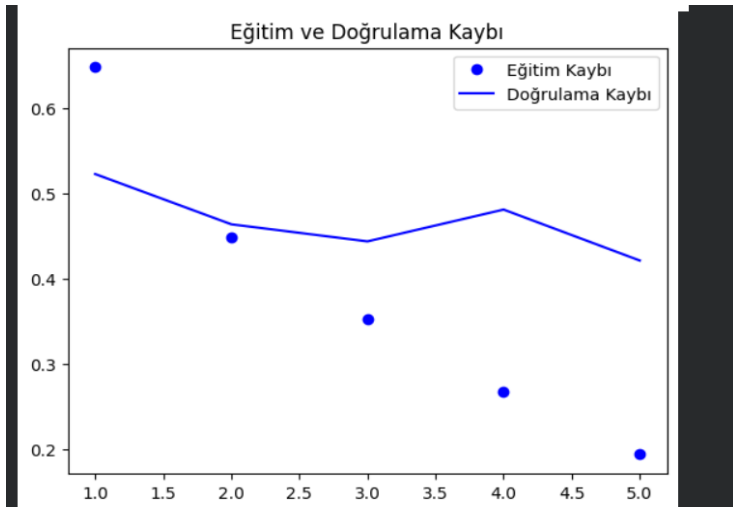
Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
simple_rnn_2 (SimpleRNN)	?	0 (unbuilt)
dense_2 (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)  
Trainable params: 0 (0.00 B)  
Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/5  
157/157 ————— 33s 195ms/step - acc: 0.6057 - loss: 0.6477 - val_acc: 0.7438 - val_loss: 0.5225  
Epoch 2/5  
157/157 ————— 31s 201ms/step - acc: 0.8006 - loss: 0.4484 - val_acc: 0.8052 - val_loss: 0.4636  
Epoch 3/5  
157/157 ————— 30s 192ms/step - acc: 0.8519 - loss: 0.3520 - val_acc: 0.8230 - val_loss: 0.4435  
Epoch 4/5  
157/157 ————— 30s 192ms/step - acc: 0.8911 - loss: 0.2677 - val_acc: 0.7830 - val_loss: 0.4808  
Epoch 5/5  
157/157 ————— 41s 193ms/step - acc: 0.9263 - loss: 0.1936 - val_acc: 0.8418 - val_loss: 0.4211
```



Şekildeki grafikte görüldüğü üzere, modelin öğrenme eğrisi istikrarlı bir yükseliş sergilemiş ve doğrulama başarısı eğitim başarısı ile paralel hareket etmiştir.



Şekildeki kayıp grafiği incelendiğinde, eğitim kaybının %19 seviyelerine kadar düştüğü, doğrulama kaybının ise makul bir düzeyde kalarak modelin kararlı çalıştığını kanıtladığı görülmektedir.

SONUÇ:

Bu proje ile Yinelemeli Sinir Ağlarının (RNN) ardışık verileri işleme yeteneği ve many-to-one mimarisi uygulamalı olarak kavranmıştır.

IMDB veri seti üzerinde yapılan testlerde %84,18 başarı oranına ulaşılması, modelin duygu analizi konusundaki etkinliğini ortaya koymuştur.

Çalışma sırasında gradyan problemleri ve veri ön işleme süreçlerinin model performansı üzerindeki kritik etkileri gözlemlenmiştir.

KAYNAKÇA

Chollet, F. (2017). *Deep Learning with Python*. Manning Publications.

Maas, A. L., et al. (2011). "Learning Word Vectors for Sentiment Analysis." *ACL*.

Keras Documentation. "Working with RNNs". https://keras.io/guides/working_with_rnn/

TensorFlow Core. "Text classification with an RNN".
https://www.tensorflow.org/tutorials/text/text_classification_rnn